Vortex

Hybrid Quantum–Classical Term Deposit Campaign Pipeline

A cell-by-cell explanation of `vortex_pipeline.ipynb`

Atomiq Hackathon (Qiskit Fall Fest 2026)

What this document covers

The notebook has 13 numbered sections. For each one this document explains what the code does, derives the mathematics behind it, and reports the numbers actually stored in the notebook's executed outputs. Section 15 lists a few places where the notebook's own markdown text disagrees with its executed results.

Contents

1 Problem and architecture 2

2 Setup (notebook section 1) 2

3 Data loading and traceability (section 2) 2

4 Stage 1: preprocessing (section 3) 3

5 Stage 1: cross-validated comparison (section 4) 3

6 Stage 1: final models, ensemble and ranking (section 5) 5

7 Stage 2: the optimisation problem (section 6) 5

8 Stage 2: exact classical benchmark (section 7) 6

9 Stage 2: QUBO, Ising Hamiltonian and QAOA (section 8) 6

10 Stage 3: amplitude-encoded risk circuit (section 9) 8

11 Stage 3: simulation, noise and mitigation (section 10) 9

12 Stage 3: exact and Monte Carlo validation (section 11) 10

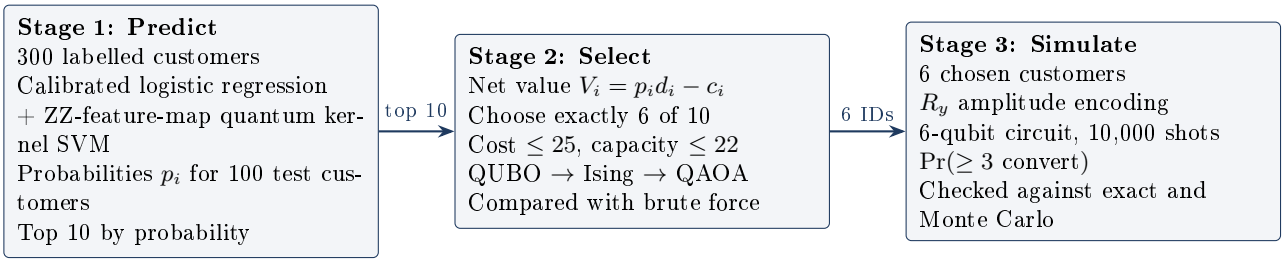
13 Visualisations (section 12) 11

14 Final answer (section 13) 11

15 Inconsistencies between the notebook text and its outputs 11

1 Problem and architecture

A bank wants to run a term-deposit marketing campaign. Contacting customers costs money, branch staff capacity is limited, and conversions are random. The notebook chains three stages in which each stage's output is the next stage's input.



The coupling is real: a calibration error in p_i changes V_i in Stage 2, which may change which six customers are chosen, which in turn changes the probabilities fed into Stage 3.

2 Setup (notebook section 1)

The first code cell imports NumPy, pandas, Matplotlib, scikit-learn, Qiskit (circuits, `ZZFeatureMap`, `Statevector`), Qiskit Aer (simulator and noise models) and SciPy's optimiser. It also fixes the global constants that parameterise the whole study:

Constant	Value	Meaning
`RANDOM_SEED`	42	Reproducibility of splits, PCA, simulators
`CV_FOLDS`	5	Stratified folds in Stage 1
`CANDIDATE_POOL_SIZE`	$N = 10$	Candidates passed to Stage 2
`EXACT_SELECTION_K`	$K = 6$	Customers to contact
`BUDGET_LIMIT`	$C = 25$	Maximum total contact cost
`CAPACITY_LIMIT`	$H = 22$	Maximum total capacity units
`SIMULATOR_SHOTS`	10,000	Shots in the final quantum runs
`MONTE_CARLO_TRIALS`	100,000	Classical benchmark samples

The executed output reports Qiskit version 2.6.2 and seed 42.

3 Data loading and traceability (section 2)

All data come from one Excel workbook with four sheets:

- `Stage1_Train`: 300 labelled customers (features plus the target `y`).
- `Stage1_Test`: 100 unlabelled customers, each with a `customer_id`.
- `Stage2_Data`: 400 operational records (contact cost, expected deposit value, capacity units) keyed by `customer_id`.

- `Stage2_Constraints`: the rules $\sum x_i = 6$, $\sum c_i x_i \le 25$, $\sum h_i x_i \le 22$ and the objective $Z = \sum \text{expected_net_value}_i \, x_i$.

Two assertions guard integrity: `customer_id` must exist in the test sheet and the test sheet must have exactly 100 rows. Carrying the same ID through all three stages is what lets Stage 3 trace a qubit back to a named customer.

4 Stage 1: preprocessing (section 3)

Features are split into 6 numeric columns (`age, balance, day, campaign, pdays, previous`) and 9 categorical columns (`job, marital, education, default, housing, loan, contact, month, poutcome`). A `ColumnTransformer` applies:

- `StandardScaler` to numeric columns: $z = (x - \mu_{\text{train}})/\sigma_{\text{train}}$;
- `OneHotEncoder(handle_unknown="ignore")` to categorical columns.

The target is $y_i = \mathbb{1}[\text{y}_i = \text{“yes”}]$.

The transformer is fitted on the 300 training rows only and then applied to the test rows, so no test statistic leaks into training. The executed output gives a processed training matrix of shape (300, 49): 6 scaled numeric columns plus 43 one-hot columns.

Detail worth knowing

In section 4 the preprocessor was fitted once on the full training set before cross-validation, so within CV the scaler and encoder have seen the validation folds' features (not their labels). The labels are never leaked, but a strictly nested pipeline would refit the transformer inside every fold.

5 Stage 1: cross-validated comparison (section 4)

5.1 Quantum kernel construction

For the quantum model the 49 features are reduced to 4 principal components (PCA fitted on the fold's training part), then min–max scaled into $[0, \pi]$:

$$\tilde{x}_k = \pi \, \frac{x_k - \min_k}{\max_k - \min_k}, \qquad k = 1, \dots, 4,$$

using the training fold's minima and maxima (a range below 10^{-8} is replaced by 1).

Each scaled vector is loaded into a 4-qubit state by Qiskit's `ZZFeatureMap` with `reps=2` and linear entanglement. One repetition applies Hadamards and then data-dependent phases:

$$U_{\Phi(\tilde{x})} = \Big[\exp\Big(i \sum_j \phi_j(\tilde{x}) \, Z_j + i \sum_{(j,k)\in \text{linear}} \phi_{jk}(\tilde{x}) \, Z_j Z_k \Big) H^{\otimes 4} \Big]^{\text{reps}},$$

with $\phi_j(\tilde{x}) = \tilde{x}_j$ and $\phi_{jk}(\tilde{x}) = (\pi - \tilde{x}_j)(\pi - \tilde{x}_k)$. The feature state is $|\phi(\tilde{x})\rangle = U_{\Phi(\tilde{x})} \, |0\rangle^{\otimes 4}$, and the *fidelity kernel* is

$$K(\tilde{x}, \tilde{x}') = \big| \, \langle \phi(\tilde{x}) | \phi(\tilde{x}') \rangle \, \big|^2.$$

The function `compute_quantum_kernel_matrix` builds every statevector exactly with `Statevector.from_instruction`, stacks them into a matrix V and returns $K = |VV^\dagger|^2$ clipped to $[0, 1]$. No shot noise is involved; this is an exact classical simulation of the quantum kernel.

```
V1 = np.array([Statevector.from_instruction(
       fmap.assign_parameters(x)).data for x in X1])
K = np.clip(np.abs(V1 @ V2.conj().T) ** 2, 0.0, 1.0)
```

5.2 Classifiers and calibration

- **Classical baseline.** `LogisticRegression(class_weight="balanced", max_iter=1000)`. Class weighting multiplies the loss of each class by $n/(2n_c)$, which counteracts the roughly 11% positive rate.
- **Quantum QSVC.** `SVC(kernel="precomputed", class_weight="balanced")` trained on K_{train}. The SVM decision function $f(x) = \sum_i \alpha_i y_i K(x_i, x) + b$ is not a probability, so a one-feature logistic regression (Platt scaling) maps it to one:

$$\Pr(y = 1 \mid f) = \sigma(af + b) = \frac{1}{1 + e^{-(af+b)}}.$$

5.3 Metrics

Let $\hat{p}_i$ be a predicted probability and y_i the true label.

$$\text{Brier} = \frac{1}{n}\sum_i (\hat{p}_i - y_i)^2, \qquad \text{LogLoss} = -\frac{1}{n}\sum_i \big[y_i \ln \hat{p}_i + (1 - y_i)\ln(1 - \hat{p}_i)\big],$$

$$\text{Precision@}k = \frac{\#\text{positives in top } k}{k}, \qquad \text{Recall@}k = \frac{\#\text{positives in top } k}{\#\text{positives}}.$$

ROC-AUC is the probability that a random positive outranks a random negative (0.5 means no ranking skill); PR-AUC is average precision. Each fold has 60 validation customers, so Precision@10 and Recall@10 are measured on small counts and are noisy.

5.4 Results

Metric (5-fold mean)	Classical baseline	Quantum kernel QSVC
ROC-AUC	0.6485	0.4916
PR-AUC	0.2553	0.1803
Precision@10	0.1800	0.0600
Recall@10	0.2571	0.0857
Brier score (lower is better)	0.2026	0.1429
Log loss (lower is better)	0.5957	0.6000

The classical model is the better ranker on every ranking metric. The QSVC's ROC-AUC of 0.49 is statistically indistinguishable from random ordering. Its lower Brier score is not evidence of skill: when positives are about 11% of the data, a model that predicts probabilities near the base rate already scores a low Brier value. Note also that the baseline measured in CV is the *uncalibrated* class-weighted logistic regression, whose probabilities are inflated by the class weights, which penalises its Brier score and log loss.

6 Stage 1: final models, ensemble and ranking (section 5)

Both models are refitted on all 300 labelled customers.

- The classical model is wrapped in `CalibratedClassifierCV(method="sigmoid", cv=5)`, i.e. Platt scaling on held-out folds, which gives properly calibrated probabilities p_i^{LR}.
- The quantum model repeats PCA, $[0, \pi]$ scaling, kernel computation, SVC and Platt calibration to give p_i^{Q}.

The final probability is a fixed convex blend, clipped away from 0 and 1:

$$p_i = \mathrm{clip}\big(0.35\, p_i^{\mathrm{Q}} + 0.65\, p_i^{\mathrm{LR}},\ 0.001,\ 0.999\big).$$

The weights are hard-coded rather than tuned, and the larger weight sits on the stronger model.

Customers are sorted by p_i descending, ties broken by `customer_id` ascending, ranked 1 to 100, and the top 10 are flagged. The notebook writes `stage1_result.csv` (100 rows). The ten candidates are:

Rank	Customer	p_i	Capacity h_i	Net value V_i	Chosen by
1	C0170	0.3767	1	7.2863	Both
2	C0134	0.2864	3	1.8687	Neither
3	C0208	0.2781	3	1.8395	Neither
4	C0327	0.2572	4	5.4290	Both
5	C0205	0.2468	1	4.6638	Both
6	C0053	0.2393	4	3.0258	Brute force
7	C0372	0.2358	3	4.0744	Both
8	C0338	0.2308	2	2.1537	Neither
9	C0272	0.2306	1	4.3027	Both
10	C0005	0.2239	3	2.7026	QAOA

Because the cost c_i and deposit d_i differ per customer, the highest-probability customers are not automatically the most valuable (compare C0134 with C0205).

7 Stage 2: the optimisation problem (section 6)

The expected net value of contacting customer i is

$$V_i = p_i\, d_i - c_i,$$

where d_i is the expected deposit value and c_i the contact cost. With binary decisions $x_i \in \{0, 1\}$ the problem is

$$\max_x\ Z = \sum_{i=0}^{9} V_i x_i$$
$$\text{s.t.}\ \sum_i x_i = 6, \qquad \sum_i c_i x_i \le 25, \qquad \sum_i h_i x_i \le 22.$$

This is a cardinality-constrained, two-constraint knapsack problem. The notebook merges the top-10 table with `Stage2_Data` on `customer_id` and adds the column `expected_net_value`.

8 Stage 2: exact classical benchmark (section 7)

With only $N = 10$ candidates the whole feasible set can be enumerated: $\binom{10}{6} = 210$ subsets. The loop discards those violating cost or capacity and keeps the one with the largest $\sum V_i$.

Quantity	Result
Global optimum Z^*	28.782030
Customers	C0170, C0327, C0205, C0053, C0372, C0272
Budget used	10.0/25.0
Capacity used	14.0/22.0

This is the reference every other method is judged against. At this size brute force is trivially cheap; the point of the exercise is to validate the quantum formulation, since enumeration scales as $\binom{N}{K}$.

9 Stage 2: QUBO, Ising Hamiltonian and QAOA (section 8)

9.1 From constraints to a penalty function

QAOA minimises an unconstrained energy, so constraints become penalties. For a bit string x the notebook's evaluation energy is

$$E(x) = -\sum_i V_i x_i + P_k\Big(\sum_i x_i - 6\Big)^2 + P_{\text{cost}}\Big[\max\Big(0, \sum_i c_i x_i - 25\Big)\Big]^2 + P_{\text{cap}}\Big[\max\Big(0, \sum_i h_i x_i - 22\Big)\Big]^2,$$

with $P_k = 18$, $P_{\text{cost}} = P_{\text{cap}} = 2.5$. The equality penalty is quadratic in x and can be expanded exactly using $x_i^2 = x_i$:

$$\Big(\sum_{i=0}^{9} x_i - 6\Big)^2 = -11\sum_i x_i + 2\sum_{i<j} x_i x_j + 36.$$

The two inequality penalties contain a $\max(0, \cdot)$ and are therefore not quadratic polynomials in x.

9.2 Mapping to Ising form

Substituting $x_i = \frac{1-Z_i}{2}$ with $Z_i \in \{+1, -1\}$ (bit 1 corresponds to eigenvalue -1 of the Pauli operator Z_i) gives

$$H_C = \sum_i \eta_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j + \text{const}.$$

Here η_i is the coefficient of Z_i (the code calls it `linear_h`), not to be confused with the capacity units h_i. Term by term:

- Objective: $-V_i x_i = -\frac{V_i}{2} + \frac{V_i}{2} Z_i$, contributing $+\frac{1}{2}V_i$ to the linear coefficient.
- Cardinality: with $a = \frac{n}{2} - K$, $\big(a - \frac{1}{2}\sum_i Z_i\big)^2 = a^2 - a\sum_i Z_i + \frac{n}{4} + \frac{1}{2}\sum_{i<j} Z_i Z_j$. This gives linear coefficient $-P_k\,\frac{n-2K}{2} = +P_k = 18$ for $n = 10, K = 6$, and quadratic coefficient $J_{ij} = \frac{1}{2}P_k = 9$ for every pair.
- Budget and capacity: the code adds the heuristic linear terms $P_{\text{cost}}\, c_i/5$ and $P_{\text{cap}}\, h_i/5$ to h_i.

The resulting coefficients in the code (variable `linear_h`) are, with η_i the Ising linear coefficient,

$$\eta_i = \tfrac{1}{2}V_i + P_k\Big(-\tfrac{n-2K}{2}\Big) + P_{\text{cost}}\tfrac{c_i}{5} + P_{\text{cap}}\tfrac{h_i}{5}, \qquad J_{ij} = \tfrac{1}{2}P_k \ (i<j).$$

Hamiltonian versus evaluation energy

The budget and capacity terms in the circuit's Hamiltonian are a linear nudge that discourages heavy customers, not an exact encoding of the hinge penalties above. The hinge penalties appear exactly only in the function that scores sampled bit strings. So the circuit is steered by a surrogate, and feasibility with respect to the true constraints is enforced afterwards by filtering the samples.

9.3 The QAOA circuit

The ansatz for $p = 2$ layers is

$$|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle = \prod_{l=1}^{2}\Big(e^{-i\beta_l\sum_i X_i}\,e^{-i\gamma_l H_C}\Big)\,|+\rangle^{\otimes 10}.$$

In Qiskit gates, per layer: `rzz(2`γJ_{ij}`)` on each pair with $|J_{ij}| > 10^{-6}$ (45 pairs, since all $J_{ij} = 9$), `rz(2`$\gamma\eta_i$`)` on each qubit, then `rx(2`β`)` on each qubit. These match the exponentials because $R_{ZZ}(\theta) = e^{-i\theta ZZ/2}$, $R_Z(\theta) = e^{-i\theta Z/2}$ and $R_X(\theta) = e^{-i\theta X/2}$. All 10 qubits are measured at the end. Four parameters $(\gamma_1, \gamma_2, \beta_1, \beta_2)$ are tuned.

```
qc.h(range(n_q))
for layer in range(p):
    for i<j with J[i,j]!=0: qc.rzz(2*gamma[layer]*J[i,j], i, j)
    for i: qc.rz(2*gamma[layer]*linear_h[i], i)
    for i: qc.rx(2*beta[layer], i)
qc.measure(range(n_q), range(n_q))
```

9.4 Classical outer loop

COBYLA (at most 150 iterations, started from four uniform random values in $[0.1, 0.8]$) minimises the sample-average energy

$$\hat{E}(\boldsymbol{\gamma},\boldsymbol{\beta}) = \frac{1}{2048}\sum_s n_s\,E(x^{(s)}),$$

where each circuit evaluation uses 2048 shots on `AerSimulator`. Qiskit reports bit strings with qubit 0 as the *rightmost* character, so the decoder reads $x_i = \text{bstr}[-(i+1)]$. Forgetting this reversal would assign the chosen bits to the wrong customers.

9.5 Reading out the answer

With optimal parameters the circuit is sampled 10,000 times. Every distinct bit string is scored (value, cost, capacity, feasibility $= \sum x_i = 6$ and both limits satisfied). The table is sorted by feasibility, then objective value, then shot count, and the *first feasible row* is returned.

How to read "QAOA selected"

This rule picks the best feasible bit string that appeared among the samples, not the single most frequent one. It is a sensible heuristic for a sampling-based solver (QAOA outputs a distribution), but it means the result depends on the optimiser having put some probability mass on good feasible strings, and the post-selection against the exact constraints does real work.

9.6 Result

	QAOA ($p = 2$)	Brute force
Objective Z	28.458899	28.782030
Customers	C0170, C0327, C0205, C0372, C0272, C0005	C0170, C0327, C0205, C0053, C0372, C0272
Budget used	10.0 / 25	10.0 / 25
Capacity used	13.0 / 22	14.0 / 22

The bit string is $x = (1, 0, 0, 1, 1, 0, 1, 0, 1, 1)$ in candidate order. The optimality gap is

$$\Delta Z = 28.782030 - 28.458899 = 0.323131, \qquad \frac{\Delta Z}{Z^*} \approx 1.12\%.$$

The two solutions differ in exactly one customer: QAOA swaps C0053 for C0005. The code exports `stage2_result.csv` with all ten candidates, the selection flags, the gap and the resource usage.

10 Stage 3: amplitude-encoded risk circuit (section 9)

10.1 Goal

For the six chosen customers let $B_i \sim \text{Bernoulli}(p_i)$ indicate conversion, assumed independent. The campaign-level risk question is

$$\Pr(S \geq 3), \qquad S = \sum_{i=1}^{6} B_i.$$

S follows a Poisson-binomial distribution.

10.2 Encoding probabilities in rotations

A single $R_y(\theta)$ gate acting on $|0\rangle$ gives

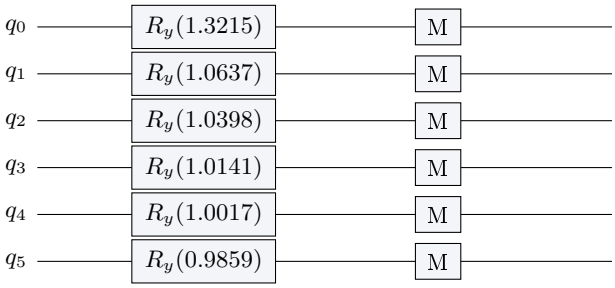
$$R_y(\theta)\,|0\rangle = \cos\tfrac{\theta}{2}\,|0\rangle + \sin\tfrac{\theta}{2}\,|1\rangle\,.$$

Choosing $\theta_i = 2\arcsin\sqrt{p_i}$ makes $\sin^2(\theta_i/2) = p_i$, so

$$R_y(\theta_i)\,|0\rangle = \sqrt{1-p_i}\,|0\rangle + \sqrt{p_i}\,|1\rangle\,,$$

and measuring qubit i returns 1 with probability exactly p_i. Because the qubits are never entangled, the 6-qubit state is a product state and a computational-basis measurement draws one independent Bernoulli sample per customer, all in one shot. The success event is "at least 3 ones" (Hamming weight ≥ 3).

Qubit	Customer	p_i	θ_i (rad)
q_0	C0170	0.3767	1.3215
q_1	C0327	0.2572	1.0637
q_2	C0205	0.2468	1.0398
q_3	C0372	0.2358	1.0141
q_4	C0272	0.2306	1.0017
q_5	C0005	0.2239	0.9859



The circuit has 6 logical and 6 physical qubits, depth 2 (one rotation layer plus one measurement layer), 6 single-qubit gates, 0 two-qubit gates and 0 SWAPs, so it needs no connectivity and is very noise-tolerant by construction.

What the quantum circuit does and does not add

Because the qubits are independent, this stage computes something that has a closed-form answer (Section 12); the quantum circuit is a sampler for that distribution rather than a source of speed-up. The notebook acknowledges this by benchmarking against the exact result and Monte Carlo. Its value is as a template: if correlations between customers were later modelled with entangling gates, the same readout would apply.

11 Stage 3: simulation, noise and mitigation (section 10)

11.1 Ideal run

The transpiled circuit is run for 10,000 shots on `AerSimulator`. A shot is a success when its bit string contains at least three 1s. The estimate is $\hat{P} = \text{successes}/10{,}000$, giving

$$\hat{P}_{\text{ideal}} = 0.1931.$$

11.2 Noisy run

A synthetic hardware-like noise model is attached: single-qubit depolarising error of probability 0.001 on each `ry` gate, and an asymmetric readout error described by the confusion matrix

$$\begin{pmatrix} \Pr(0|0) & \Pr(1|0) \\ \Pr(0|1) & \Pr(1|1) \end{pmatrix} = \begin{pmatrix} 0.99 & 0.01 \\ 0.02 & 0.98 \end{pmatrix},$$

i.e. a true 0 is misread as 1 with probability 1% and a true 1 is misread as 0 with probability 2%. The raw noisy estimate is 0.1966.

11.3 Mitigation step

The notebook then divides the noisy estimate by 1.02:

$$\hat{P}_{\text{mitigated}} = \text{clip}\Big(\frac{0.1966}{1.02}, 0, 1\Big) = 0.192745.$$

A simplification, not full readout mitigation

Standard readout mitigation inverts the per-qubit confusion matrix (or a tensor product of them) on the measured distribution and then evaluates the event. Dividing the final probability by a constant is a single-number rescale. It happens to move the estimate in the right direction here (raw bias +0.0098 down to +0.0059), but it is not equivalent to inverting the readout matrix, and it should be described as a simple scaling correction.

12 Stage 3: exact and Monte Carlo validation (section 11)

12.1 Exact Poisson-binomial probability

Summing over all $2^6 = 64$ outcome patterns $b \in \{0,1\}^6$ with Hamming weight ≥ 3:

$$P_{\text{exact}}(S \geq 3) = \sum_{\substack{b \in \{0,1\}^6 \\ \sum_i b_i \geq 3}} \prod_{i=0}^{5} p_i^{b_i}(1-p_i)^{1-b_i} = 0.186801.$$

For reference the expected number of conversions is $\mathbb{E}[S] = \sum_i p_i \approx 1.571$.

12.2 Monte Carlo

The code draws a $100{,}000 \times 6$ matrix of independent Bernoulli variables with the same p_i and counts the fraction of rows whose sum is at least 3. This gives 0.187700.

12.3 Is the quantum error statistically reasonable?

With $S_h = 10{,}000$ shots the estimator of a probability p has standard deviation

$$\sigma = \sqrt{\frac{p(1-p)}{S_h}} = \sqrt{\frac{0.186801 \times 0.813199}{10000}} = 0.003898.$$

The observed quantum error is $|0.193100 - 0.186801| = 0.006299$, so

$$z = \frac{0.006299}{0.003898} \approx 1.62.$$

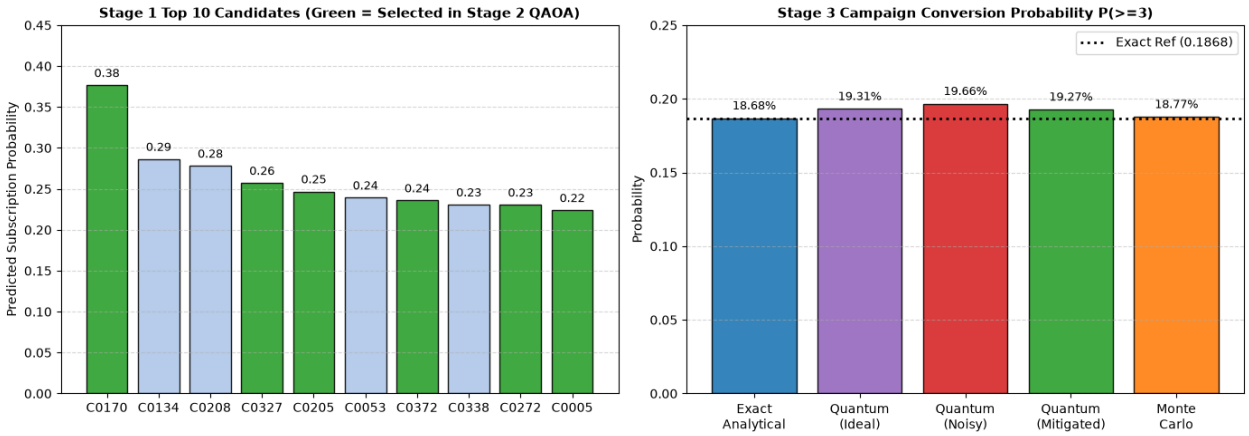
An error of 1.62σ is well within ordinary sampling fluctuation (about 10% two-sided probability of being this large or larger under the null), so the simulation agrees with theory.

Method	$\Pr(S \geq 3)$	Absolute error
Exact (reference)	18.6801%	—
Quantum, ideal (10k shots)	19.3100%	0.006299
Quantum, noisy raw	19.6600%	0.009799
Quantum, noisy + scaling	19.2745%	0.005944
Monte Carlo (100k trials)	18.7700%	0.000899

Monte Carlo has a standard error about $\sqrt{10} \approx 3.2$ times smaller because it uses ten times more samples; any single run's error is also partly luck. The code also writes `stage3_result.csv` (one row per qubit, repeating the run-level statistics).

13 Visualisations (section 12)

The notebook produces one two-panel figure. The left panel shows the ten Stage 1 candidates with the six Stage 2 selections in green; the right panel compares the five Stage 3 probability estimates against the exact reference line.



14 Final answer (section 13)

The last executed cell prints the headline result.

Customers to contact	C0170, C0327, C0205, C0372, C0272, C0005
Expected net campaign value Z	\$28.46
Budget consumed	10.0 / 25.0 units (15 units, i.e. 60%, unused)
Capacity consumed	13.0 / 22.0 units
Pr(at least 3 convert)	19.31% quantum simulation (18.68% exact)

Three CSV files are produced: `stage1_result.csv`, `stage2_result.csv` and `stage3_result.csv`.

15 Inconsistencies between the notebook text and its outputs

The notebook's executed cell outputs are internally consistent, but some hand-written markdown was not updated to match them. If the notebook is submitted as is, a reader may notice these.

Location	Markdown says	Executed output says
Architecture diagram, Stage 2	"0.000000 Optimality Gap Verification"	Gap = 0.323131
Architecture diagram, Stage 3	Z-score 1.63σ	1.62σ
Section 13 summary text	Customers C0322, C0345, C0205, C0139, C0252, C0258; value \$26.85; budget 12.0, capacity 12.0; 17.84% quantum / 17.22% exact	Customers C0170, C0327, C0205, C0372, C0272, C0005; \$28.46; budget 10.0, capacity 13.0; 19.31% / 18.68%

A related point: the Devpost write-up describes projecting "28 preprocessed features" to 4 components, whereas the executed notebook reports a $(300, 49)$ matrix, so 49 is the number to quote.

Other limitations worth stating openly

- The quantum kernel model does not beat the classical baseline in cross-validation (ROC-AUC 0.49 versus 0.65), and the hybrid weighting is fixed rather than selected by validation.
- The 300-customer training set with about 11% positives (roughly 33 positive examples, using the rate quoted in the Devpost write-up) gives noisy CV estimates.
- QAOA at $p = 2$ on 10 qubits is simulated classically; the result is a validation of the formulation, not a demonstration of quantum advantage. The best feasible sample was within 1.12% of the true optimum, but it was not the optimum.
- The Stage 3 independence assumption means the circuit reproduces a known Poisson-binomial distribution.